

Distributed [*Computing*] Systems

Matrix Multiplication

Isaac D. Scherson (aka The Schark 😊)

Dept. of Computer Science (Systems)
Bren School of Information and Computer Sciences
University of California, Irvine
Irvine, CA 92697-3425

isaac@ics.uci.edu
www.ics.uci.edu/~isaac www.ics.uci.edu/~schark

CompSci-230, Winter 2019



© Isaac D. Scherson

1 / 19

Matrix Multiplication



© Isaac D. Scherson

2 / 19

Problem Statement

- ▶ Consider the problem of computing the product of two $n \times n$ complex matrices A and B
- ▶ The problem is to be solved on a n -processor ring.

$$A = \begin{bmatrix} a_{00} & a_{01} & \dots & a_{0(n-1)} \\ a_{10} & a_{11} & \dots & a_{1(n-1)} \\ \vdots & & & \vdots \\ a_{(n-1)0} & a_{(n-1)1} & \dots & a_{(n-1)(n-1)} \end{bmatrix} \quad C = A \times B$$
$$B = \begin{bmatrix} b_{00} & b_{01} & \dots & b_{0(n-1)} \\ b_{10} & b_{11} & \dots & b_{1(n-1)} \\ \vdots & & & \vdots \\ b_{(n-1)0} & b_{(n-1)1} & \dots & b_{(n-1)(n-1)} \end{bmatrix}$$

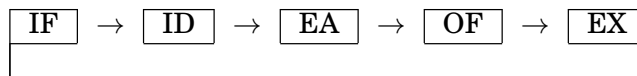


The power of a simple computation

BEFORE ...

Way back when, computer architects designed CPUs with a simple accumulator oriented operation in a memory oriented Von Neumann computer:

Recall the Von Neumann Cycle:



If a is the accumulator, b is a CPU register, and $M[EA]$ is the location in memory where we find an operand, the basic operation is:

$$a \leftarrow M[EA] + a \times b$$



The power of a simple computation

Consider the problem of computing the value of the polynomial

$$P(x) = \alpha_{n-1}x^{n-1} + \alpha_{n-2}x^{n-2} + \dots + \alpha_1x + \alpha_0$$

at $x = x_0$, without computing the powers of x .



Solution using the power of a simple computation

The polynomial evaluation is obtained by substituting x by x_0

$$P(x_0) = \alpha_{n-1}x_0^{n-1} + \alpha_{n-2}x_0^{n-2} + \dots + \alpha_1x_0 + \alpha_0$$

and the following factorization allows the evaluation without computing the powers of x :

$$P(x_0) = (\dots((\alpha_{n-1}x_0 + \alpha_{n-2})x_0 + \alpha_{n-3})x_0 + \dots)x_0 + \alpha_0$$

Code:

```
P = 0
For i = 0, n - 1, do:
    P = P × x0 + αn-1-i
endfor
```

Referring to the Powerful basic computation, $M[EA]$ contains the array of coefficients, b contains x_0 and P is assigned to the accumulator.

This polynomial evaluation method is known as Horner's rule.



Basic Sequential Nested Loop

Back to Matrix Multiplication

On a single processor system:

```
For  $i = 0, n - 1$ , do:  
  For  $j = 0, n - 1$ , do:  
    For  $k = 0, n - 1$ , do:  
       $c_{i,j} = c_{i,j} + a_{i,k} \times b_{k,j}$   
    endfor  
  endfor  
endfor
```

Complexity

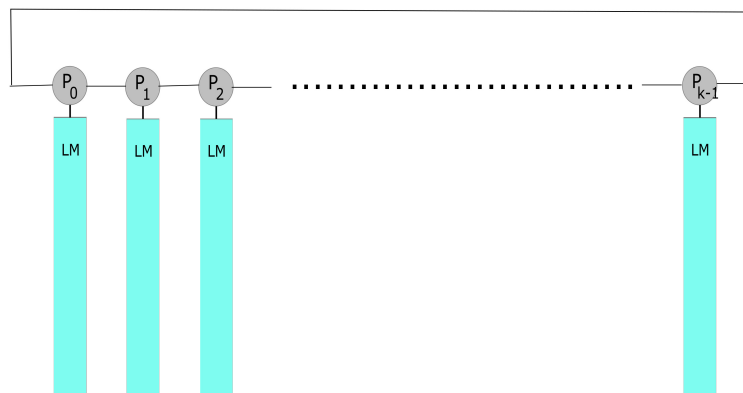
```
Time =  $O(n^3)$  Multiplications  
Memory =  $O(n^2)$   
Communications = 0
```

Note that the expression $c_{i,j} = c_{i,j} + a_{i,k} \times b_{k,j}$ is the powerful simple computation, which can be used to do the Scalar Product of two vectors. In this case, the dot product of row vectors of A and column vectors of B .



Map Matrix Multiply to a Parallel Architecture

The Architecture: n -processor Ring.
For simplicity, size matches the size of the problem.



We must first decide on how to allocate the data in memory ... Just like we do in a single processor system.



Very first approximation

Store the matrices A and B in ALL local memories (global knowledge) in row major form.

For Matrix A :

P_0	P_1	P_2	$\dots$	P_{n-1}
a_{00}	a_{00}	a_{00}	$\dots\dots$	a_{00}
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$a_{0(n-1)}$	$a_{0(n-1)}$	$a_{0(n-1)}$	$\dots\dots$	$a_{0(n-1)}$
a_{10}	a_{10}	a_{00}	$\dots\dots$	a_{10}
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$a_{1(n-1)}$	$a_{1(n-1)}$	$a_{1(n-1)}$	$\dots\dots$	$a_{1(n-1)}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$a_{(n-1)0}$	$a_{(n-1)0}$	$a_{(n-1)0}$	$\dots\dots$	$a_{(n-1)0}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$a_{(n-1)(n-1)}$	$a_{(n-1)(n-1)}$	$a_{(n-1)(n-1)}$	$\dots\dots$	$a_{(n-1)(n-1)}$



Global Knowledge Algorithm

P_0	P_1	P_2	$\dots$	P_{n-1}
A	A	A	$\dots\dots$	A
B	B	B	$\dots\dots$	B
c_{00}	c_{10}	c_{20}	$\dots\dots$	$c_{(n-1)0}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$c_{0(n-1)}$	$c_{1(n-1)}$	$c_{2(n-1)}$	$\dots\dots$	$c_{(n-1)(n-1)}$

- Every Processor P_i computes the i – th row of C
- The SPMD program becomes:

```

Do All  $P_{i=0,n-1}$ :
  For  $j = 0, n - 1$ , do:
    For  $k = 0, n - 1$ , do:
       $c_{i,j} = c_{i,j} + a_{ik} \times b_{k,j}$ 
    endfor
  endfor
enddoall

```



Global Knowledge Algorithm: What did we learn?

- ▶ Parallelization occurred by *unfolding* the loop on i
- ▶ Complexity:

Time = $O(n^2)$ Multiplications Memory = $O(n^2)$ Communications = 0
--

- ▶ Used only ONE row of A at each processor, but had ALL of A available → can save storage.
- ▶ Could also store B by columns and use the interconnection network in between each *row-column* scalar multiply iteration.
- ▶ Would use the network to bring ONE column of B to each processor when needed for the current iteration.



Reducing Storage Requirements

Store the matrices A and B in by rows and columns respectively.
The result C will be obtained by rows as in the previous case.

P_0	P_1	P_2	...	P_{n-1}
a_{00}	a_{10}	a_{20}		$a_{(n-1)0}$
.	.	.		.
.	.	.		.
$a_{0(n-1)}$	$a_{1(n-1)}$	$a_{2(n-1)}$		$a_{(n-1)(n-1)}$
b_{00}	b_{01}	b_{02}		$b_{0(n-1)}$
.	.	.		.
.	.	.		.
$b_{(n-1)0}$	$b_{(n-1)1}$	$b_{(n-1)2}$		$b_{(n-1),(n-1)}$
c_{00}	c_{10}	c_{20}		$c_{(n-1)0}$
.	.	.		.
.	.	.		.
$c_{0(n-1)}$	$c_{1(n-1)}$	$c_{2(n-1)}$		$c_{(n-1)(n-1)}$



Reducing Storage Requirements

Store the matrices A and B in by rows and columns respectively.
The result C will be obtained by rows as in the previous case.

P_0	P_1	P_2	$\dots$	P_{n-1}
a_{00}	a_{10}	a_{20}	$\dots\dots$	$a_{(n-1)0}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$a_{0(n-1)}$	$a_{1(n-1)}$	$a_{2(n-1)}$	$\dots\dots$	$a_{(n-1)(n-1)}$
b_{00}	b_{01}	b_{02}	$\dots\dots$	$b_{0(n-1)}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$b_{(n-1)0}$	$b_{(n-1)1}$	$b_{(n-1)2}$	$\dots\dots$	$b_{(n-1),(n-1)}$
c_{00}	c_{10}	c_{20}	$\dots\dots$	$c_{(n-1)0}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$c_{0(n-1)}$	$c_{1(n-1)}$	$c_{2(n-1)}$	$\dots\dots$	$c_{(n-1)(n-1)}$



Mixed Row-Column Algorithm

- Every Processor P_i computes the i - th row of C
- The SPMD program becomes:

```

Do All  $P_{i=0,n-1}$ :
  For  $j = i$ , do  $n$  times:
    For  $k = 0, n - 1$ , do:
       $c_{i,j} = c_{i,j} + a_{ik} \times b_{k,j}$ 
    endfor
  Every processor  $i$  sends its current
  column of  $B$  to processor  $(i + 1) \bmod(n)$ 
  endfor
enddoall

```

- Complexity:

```

Time =  $O(n^2)$  Multiplications
Memory =  $O(n)$ 
Communications =  $O(n^2)$  network cycles

```

Assumption: Only ONE data item can be transmitted in a single network cycle.



Global knowledge and Mixed Row-Column

- ▶ In BOTH Global knowledge and Mixed Row-Column algorithms parallelization occurred by *unfolding* the loop on i
- ▶ Both exhibit the same time complexity: a speedup of n with n processors.
- ▶ Mixed Row-Column gains in storage at the expense of communications.



A Challenge !!!

Store the matrices A , B and C consistently by rows.
Devise an algorithm that works with this storage scheme.

P_0	P_1	P_2	$\dots$	P_{n-1}
a_{00}	a_{10}	a_{20}	$\dots\dots$	$a_{(n-1)0}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$a_{0(n-1)}$	$a_{1(n-1)}$	$a_{2(n-1)}$	$\dots\dots$	$a_{(n-1)(n-1)}$
b_{00}	b_{10}	b_{20}	$\dots\dots$	$b_{(n-1)0}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$b_{0(n-1)}$	$b_{1(n-1)}$	$b_{2(n-1)}$	$\dots\dots$	$b_{(n-1)(n-1)}$
c_{00}	c_{10}	c_{20}	$\dots\dots$	$c_{(n-1)0}$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$\cdot$	$\cdot$	$\cdot$		$\cdot$
$c_{0(n-1)}$	$c_{1(n-1)}$	$c_{2(n-1)}$	$\dots\dots$	$c_{(n-1)(n-1)}$



Observation

- ▶ When multiplying rows of A by columns of B , one element of the row of A multiplies ALL the elements of the corresponding row of B .

$$A = \begin{bmatrix} a_{00} & a_{01} & \dots & a_{0(n-1)} \\ a_{10} & a_{11} & \dots & a_{1(n-1)} \\ \vdots & \color{blue}{a_{ij}} & & \vdots \\ \vdots & & & \vdots \\ a_{(n-1)0} & a_{(n-1)1} & \dots & a_{(n-1)(n-1)} \end{bmatrix}$$
$$B = \begin{bmatrix} b_{00} & b_{01} & \dots & b_{0(n-1)} \\ b_{10} & b_{11} & \dots & b_{1(n-1)} \\ \vdots & & & \vdots \\ \color{blue}{b_{j0}} & \color{blue}{b_{j1}} & \dots & \color{blue}{b_{j(n-1)}} \\ \vdots & & & \vdots \\ b_{(n-1)0} & b_{(n-1)1} & \dots & b_{(n-1)(n-1)} \end{bmatrix}$$



Storage by Rows Algorithm

- ▶ Every Processor P_i computes the i – th row of C
- ▶ The SPMD program becomes:

```
Do All  $P_{i=0, n-1}$ :  
  For  $j = i$ , do  $n$  times:  
    For  $k = 0, n - 1$ , do:  
       $c_{i,j} = c_{i,j} + a_{ij} \times b_{j,k}$   
    endfor  
    Every processor  $i$  sends its current  
    row of  $B$  to processor  $(i + 1) \bmod(n)$   
  endfor  
enddoall
```

- ▶ Complexity:

```
Time =  $O(n^2)$  Multiplications  
Memory =  $O(n)$   
Communications =  $O(n^2)$  network cycles
```

Assumption: Only ONE data item can be transmitted in a single network cycle.



Another Challenge

```
For  $i = 0, n - 1$ , do:  
  For  $j = 0, n - 1$ , do:  
    For  $k = 0, n - 1$ , do:  
       $c_{i,j} = c_{i,j} + a_{ik} \times b_{k,j}$   
    endfor  
  endfor  
endfor
```

- ▶ Starting with the basic sequential loop, the loop on i was unfolded to expose the data parallelism.
- ▶ Invariant program transformations allow for changing the order of the loops leaving the result unchanged.
- ▶ **QUESTION:** What algorithms result when unfolding (parallelizing) on j or k ???

